

MR3

Musterart	Begründung	Komplexität
Einfache lineare Muster z.B. (A B C), keine Quantoren)	Single-pass über Partition; konstante Arbeit pro Zeile. Es gibt keine Schleifen/Quantoren und keine Alternativen, also keine multiplen parallelen Pfade, die anwachsen könnten. Daher ist die Arbeit pro Input-Event in einer festen Konstante beschränkt	linear, schnell ($O(N)$)
Feste Wiederholung $A\{3\}$	Konstanter Zustandsraum, keine Alternativen, keine Nichtdeterministik	schnell, $O(n)$
Optionale Elemente ($A? B$)	Wenig Zustandsverzweigung. Falls A und B denselben Typ repräsentieren oder die DEFINE-Bedingungen komplex sind, kann es in Implementierungen zu einem kurzen Backtracking kommen, aber die Anzahl der Alternativen bleibt konstant	mittel, $O(n)$
Muster mit begrenzten Quantoren (z.B. $A\{1,3\}$)	Max. k Kandidaten pro Startposition. Der NFA hat eine kleine Schleife mit höchstens k Wiederholungsstufen — das ist ein endlicher (und kleine) Zustandengraph; Subset-Konstruktion oder NFA-Simulation erzeugt konstant viele aktive Zustände.	$N * k$ (linear mit Faktor),
Einfache Alternation ohne Quantor: $A B$	nur wenige feste Alternativen → keine Explosion	$O(n)$
Reluctant Wiederholung: $(A+?)$	Zusätzliche Rücksprungpunkte, es müssen mindestens 1 A gematcht werden der Matcher versucht so wenig wie möglich, testet aber immer wieder alternative längere Matches.	mittel, $O(n \cdot k)$
Reluctant: $(A^*?)$	Startet mit 0 Wiederholungen. $A^*?$ nimmt möglichst wenig A (auch 0 ist erlaubt). Alleinstehend ergibt das keine große Arbeit: engine kann 0 A wählen und sofort prüfen/melden.	meist nicht teuer
Reluctant: $(A+? B)$	Backtrackingkette. Das Muster zwingt zur minimalen Anzahl von A (mindestens 1). Bei Fehlschlag des folgenden B muss die Engine sukzessiv mehr A-Tokens „übernehmen“ und für jede Erweiterung B erneut prüfen. Angenommen ein langer Block von A-Events (Länge $\sim n$) und B kommt nie → für jeden Startindex i werden bis zu $O(n-i)$ Versuche ausgeführt → Summe $\sim O(n^2)$.	eher langsam, $O(n^2)$
Greedy: PATTERN ($A+ B$) Der Greedy-Teil nimmt so viele A wie möglich, wenn danach kein B kommt → Backtracking	Partition enthält nur A, letztes Element fehlt für B, Backtracking über die gesamte Sequenz	normalerweise schnell, Worst-Case langsam: ($O(n^2)$ möglich)
Greedy: PATTERN ($(A+ B+) C$)	Beide Seiten können matchen → viele Entscheidungen → Backtracking. Beispiel worst case: AAAAAB (C fehlt) → System probiert A+, dann scheitert bei C, dann probiert B+, usw.	langsam, ($O(n^2)$)

Musterart	Begründung	Komplexität
NOTHING + ALL ROWS PER MATCH + A*	Viele Überlappungen und leere Matches erzeugen grosse Zahl an Ergebnissen und Berechnungen.	sehr hoch
Define A AS col > 10	Für jedes Event wird nur eine einfache Spaltenprüfung gemacht. Keine Aggregation, kein Rückgriff auf vorherige Events nötig; linear zur Eventanzahl; keine Speicherung oder Berechnung über mehrere Events hinweg.	gering
DEFINE A AS SUM(col) > 100	MATCH_RECOGNIZE muss bei jeder neuen Zeile überprüfen, wie groß die Summe aller bisherigen Werte für A ist. Dafür muss die Datenbank: - sich die laufende Summe merken - die Summe zurücksetzen, wenn das Muster zurückspringen muss (Backtracking) - bei überlappenden Matches mehrere mögliche Summen gleichzeitig ausprobieren	Laufzeit abhängig von Pattern-Komplexität, eher langsam
DEFINE mit PREV: A AS val > PREV(val)	Referenz auf vorherige Zeile erhöht Vergleiche. PREV ist schneller als SUM , weil nur ein einzelner vorheriger Wert gespeichert werden muss. Langsamer als ein einfacher Vergleich , besonders bei langen Pattern-Wiederholungen (+/*) oder Backtracking.	mittel; O(n) bis O(n ²) (wenn kombiniert mit Wiederholern)
DEFINE mit FIRST/LAST: A AS val = FIRST(B.val)	kann erfordern, dass frühere Kandidaten gespeichert/verglichen werden. MATCH_RECOGNIZE: - sich den ersten B-Wert merken - bei Backtracking den gespeicherten FIRST-Wert korrigieren - bei Overlaps eventuell mehrere mögliche FIRST-Werte gleichzeitig verwalten	mittel
SUBSET (ein Event mehreren Labels): SUBSET X = (A,B)	Ein Event kann zu mehreren Labels gehören → Kombinationen wachsen exponentiell	O(2 ⁿ) (Worst-Case)
Mehrere unbounded Quantifier: PATTERN (A B* C D)	Aufteilung der Mitte in B* / D* hat viele Partitionierungsoptionen. Baue NFA: Start → (A) → (loop B) → (C) → (D) → Accept.	schwer, O(n ²)

Musterart	Begründung	Komplexität
Alternation ohne Wiederholung: PATTERN ((A B) C)	Für jedes Event entscheidet die Engine nur einmal: passt A oder passt B?, Keine unbounded Wiederholer Keine großen Alternationsbäume Keine Partitionierung Backtracking/Zustandsausbreitung minimal	O(n)
Alternation innerhalb eines Quantifiers (+ oder *): PATTERN ((A B)+)	Die Engine muss für jede Zeile entscheiden: „Matcht das als A? Oder als B? Für lange A/B-Sequenzen müssen enorm viele Kombinationen ausprobiert werden. Wenn der Match später scheitert (z. B. weil danach ein anderes Pattern folgen würde): - Die Engine muss zurückspringen - Alternativen neu durchprobieren - Wiederholungen neu anordnen	hoch, O(n) bis O(n ²).
Alternation innerhalb eines Quantifiers (+ oder *): PATTERN (A (B C)* D)	Die Alternation im Wiederholer (B C)* erzeugt viele mögliche Wiederholungslängen. MATCH_RECOGNIZE muss alle diese Varianten prüfen, bevor „D“ gematcht wird. Dadurch entstehen viele Zustände	langsam, O(n ²)
AFTER MATCH SKIP TO NEXT ROW	erzeugt Overlapping Matches, weil nach jedem gefundenen Match der nächste Startpunkt nur 1 Zeile weiter rückt.	langsam
AFTER MATCH SKIP PAST LAST ROW	Nach einem Match springt MATCH_RECOGNIZE hinter das gefundene Match → es wird nicht für jeden Startpunkt erneut begonnen.	O(n), deutlich schneller
AFTER MATCH SKIP TO FIRST X	Das System setzt nach einem Match an der ersten Zeile fort, die zu X gehört. Liegt X weit vorne im Match oder tritt häufig auf, entstehen viele überlappende Startpunkte. Das erzeugt hohe Komplexität durch vermehrtes Backtracking. Wenn X selten ist, gibt es zwar weniger Startpunkte, aber die Engine muss tatsächlich erst die Position von X ermitteln, bevor sie weiterlaufen kann.	Eher langsam, X liegt immer sehr früh → Engine startet fast immer wieder am selben Punkt; aber wenn X weit hinten liegt → wenige Overlaps
AFTER MATCH SKIP TO LAST X	Die Suche startet an der letzten Zeile des Patterns, die zu X gehört. Liegt X typischerweise nah am Matchende, reduziert dies Überlappungen → effizient. Wenn die Position von X stark schwankt (ungünstige Verteilung), entstehen viele unregelmäßige Startpunkte, was Backtracking und Komplexität erhöht.	LAST X ist schnell, wenn X immer am Ende des Matches liegt → Startpunkt wandert gleichmäßig nach rechts → kaum Overlaps Aber wenn Startpunkt unregelmäßig verteilt, dann langsam

Musterart	Begründung	Komplexität
PATTERN ((A B)* C) DEFINE A AS val > 10 B AS val < 100 C AS val = 999	1. Nicht alle Werte erfüllen A und B. → Manche Events haben 1 mögliche Rolle, andere 2 Rollen. Das führt zu einem gemischten, unregelmäßigen Entscheidungsbaum, bei dem der Matcher immer wieder: • prüfen muss, ob diese Zeile A ist • oder doch B • oder ob das Muster lieber ins * oder doch schon zu C wechseln soll. Engine probiert dann alle möglichen Varianten: • A A A ... A → kein C → Backtracking • A A A ... B → kein C → Backtracking • A A B A ... → Backtracking • ... usw.	sehr hoch
PATTERN ((A B)* C) DEFINE A AS val BETWEEN 10 AND 50 B AS val BETWEEN 30 AND 60 C AS val = 999	(A B)* → jede Zeile mehrere mögliche Klassifikationen (Overlap A&B)* → unendliche Wiederholungen C spät oder selten → NFA muss viele Pfade prüfen	langsam